🗄️ Database: MySQL on Kubernetes

Welcome to the Database component of the DevOps Portfolio! Unlike the frontend and backend, we don't write application code here. Instead, we use Kubernetes YAML files to deploy an official MySQL database image.

🛠️ Technology Stack

- **MySQL (v8):** A highly popular open-source relational database.
- **Kubernetes StatefulSet:** The K8s resource type used specifically for databases to ensure data safety.
- **Persistent Volumes (PVC):** Kubernetes storage drives that attach to the database so data isn't lost if the database crashes.

📂 The Kubernetes Configuration Files

Let's break down the YAML files in `k8s/database/`:

1. `secret.yml`

- **What it is:** A secure vault for sensitive data.
- **How it works:** We store the `mysql-root-password` here in base64 encoding. Doing this ensures we never hardcode passwords directly into our deployment files. The MySQL container reads this secret when it starts to set up the root account.

2. `config.yml` (The Initialization Script)

- **What it is:** A `ConfigMap` storing a raw `.sql` file (`init.sql`).
- **How it works:** When a MySQL container boots up for the very first time, it looks inside a special folder (`/docker-entrypoint-initdb.d/`). We use this ConfigMap to inject our `init.sql` script into that folder.
- **The Result:** The database automatically creates the `portfolio` database schema and populates it with your initial John Doe profile data!

3. `stateful-set.yml`

- **What it is:** The actual database deployment.
- **Why a StatefulSet (and not a standard Deployment)?**
  - **Predictable Names:** The pod will always be called `portfolio-mysql-0`.
  - **Storage Persistence:** It requests a 1GB `PersistentVolumeClaim`. If the pod is deleted or crashes, Kubernetes detaches the storage volume and re-attaches it to the newly created pod automatically! Data is never lost.

4. `service.yml`

- **What it is:** A "Headless" Kubernetes Service (`clusterIP: None`).
- **How it works:** Standard services load-balance network traffic randomly across pods. Since we only have one primary database pod, and databases require stable network connections for data streaming, a headless service bypasses the load-balancer and points the local DNS (`portfolio-mysql-0.mysql`) directly to the exact pod IP.

🧠 Why this matters for DevOps Students

Managing state (data) is the hardest part of Kubernetes.
Frontends and Backends are "stateless" — if a node explodes, K8s can spin up a new backend pod somewhere else and the app keeps working instantly.

Databases are "stateful". If a database pod explodes, you lose all user data. By learning how StatefulSets `#3` and Persistent Volumes work together, you learn how to protect the most valuable asset in any software company: The Data.